**Don't Embed Wrong!**

Matt Williams
85.6K subscribers

Join | Subscribe | 1.8K | Share | Ask | Download | ...

30,006 views Premiered Oct 31, 2024 #AI #Ollama #RAG

As a founding member of the Ollama team, I discovered I've been doing embeddings wrong all along - and you probably are too. In this eye-opening video, I reveal how a simple technique called "prefixing" can dramatically improve your RAG application's accuracy by up to 2x.

Learn about:
- What prefixes are and how they work
- The 3 embedding models that support prefixes
- Detailed performance comparisons across different models
- Real-world testing results and implications
- Why traditional LLMs shouldn't be used for embeddings

I've conducted extensive testing comparing 5 different embedding models, with and without prefixes, across multiple scenarios. The results will surprise you - they certainly surprised me!

https://github.com/technovangelist/videoprojects/tree/main/2024-10-29-embeddings

technovangelist / videoprojects

Type / to search

Code | Issues 7 | Pull requests 6 | Actions | Projects | Security | Insights

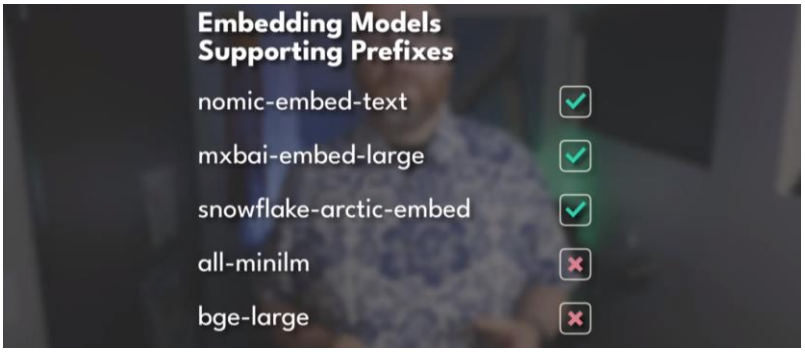
main | videoprojects / 2024-10-29-embeddings /

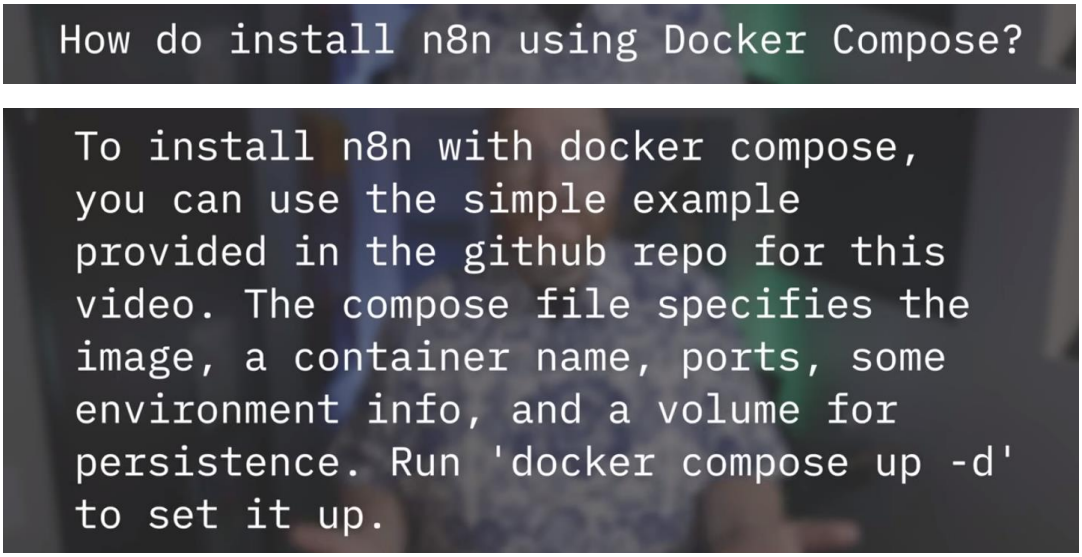
technovangelist asht

| Name | Last commit message |
|---|---|
| .. | |
| config.ts | add embed compare |
| deno.json | add embed compare |
| deno.lock | add embed compare |
| main.ts | add embed compare |
| readme.md | asht |
| test1.ts | add embed compare |
| test2.ts | add embed compare |
| test3.ts | add embed compare |

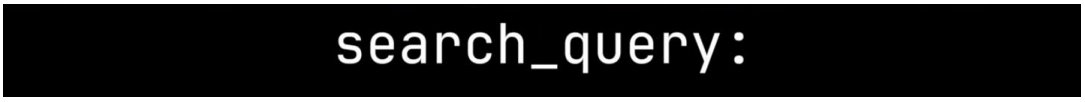



You can add a prefix to the text before sending the text for embedding, this can give 2X accuracy during retrieval





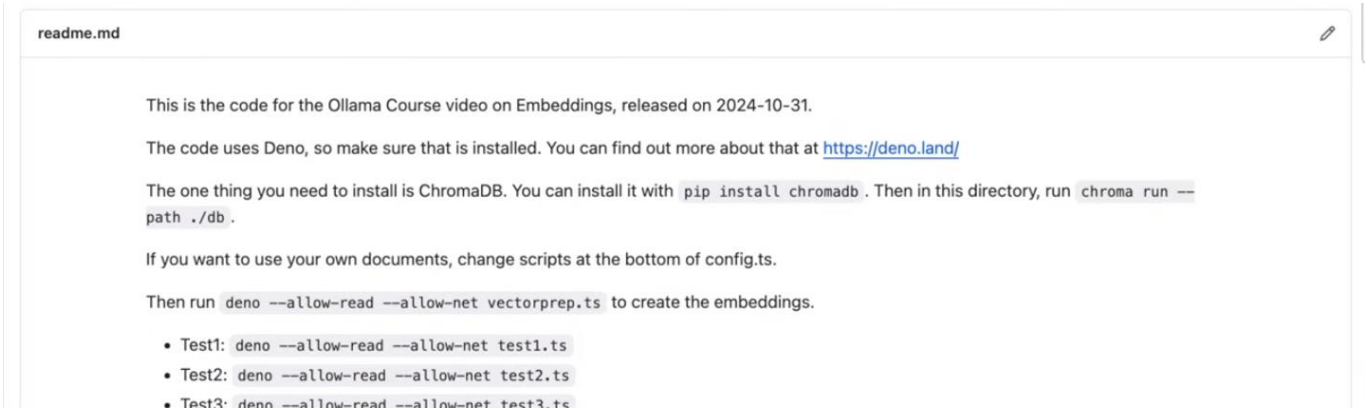

We get the above when we used prefix and nomic-embed-text model


Use the 2 prefixes above for the nomic-embed-text model for the initial chunks and the query chunks.


For classification chunks


For clustering chunks



readme.md

This is the code for the Ollama Course video on Embeddings, released on 2024-10-31.

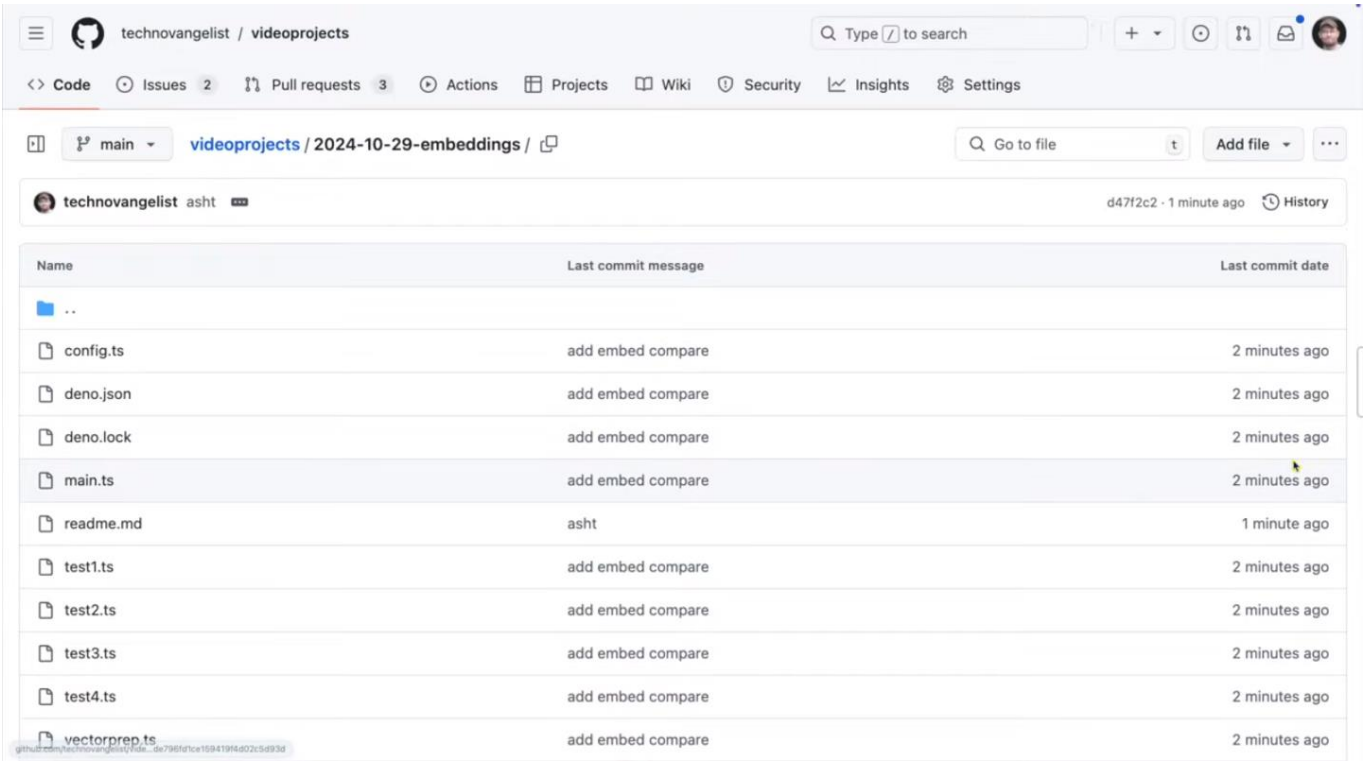
The code uses Deno, so make sure that is installed. You can find out more about that at https://deno.land/

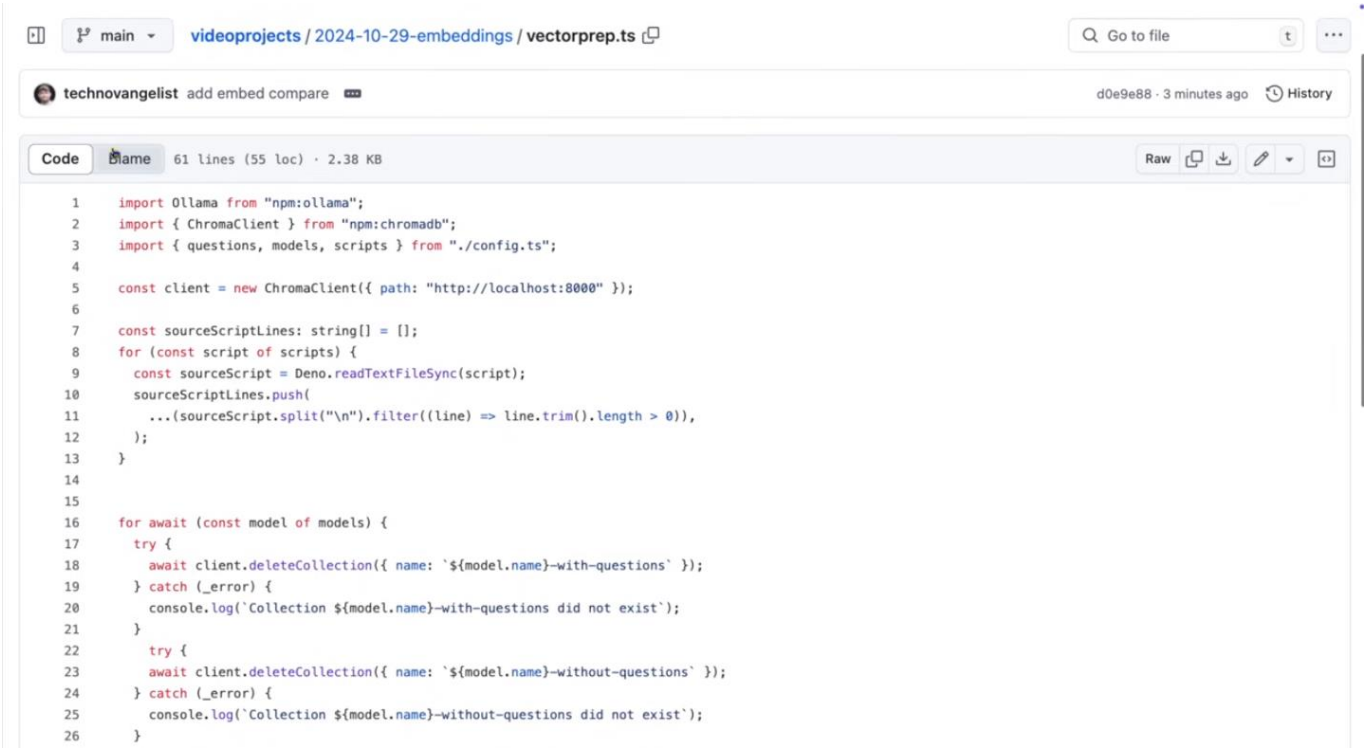
The one thing you need to install is ChromaDB. You can install it with `pip install chromadb`. Then in this directory, run `chroma run --path ./db`.

If you want to use your own documents, change scripts at the bottom of config.ts.

Then run `deno --allow-read --allow-net vectorprep.ts` to create the embeddings.

- Test1: `deno --allow-read --allow-net test1.ts`
- Test2: `deno --allow-read --allow-net test2.ts`
- Test3: `deno --allow-read --allow-net test3.ts`




```
1   import Ollama from "npm:ollama";
2   import { ChromaClient } from "npm:chromadb";
3   import { questions, models, scripts } from "./config.ts";
4
5   const client = new ChromaClient({ path: "http://localhost:8000" });
6
7   const sourceScriptLines: string[] = [];
8   for (const script of scripts) {
9     const sourceScript = Deno.readTextFileSync(script);
10    sourceScriptLines.push(
11      ...(sourceScript.split("\n").filter((line) => line.trim().length > 0)),
12    );
13  }
14
15
16  for await (const model of models) {
17    try {
18      await client.deleteCollection({ name: `${model.name}-with-questions` });
19    } catch (_error) {
20      console.log(`Collection ${model.name}-with-questions did not exist`);
21    }
22      try {
23      await client.deleteCollection({ name: `${model.name}-without-questions` });
24    } catch (_error) {
25      console.log(`Collection ${model.name}-without-questions did not exist`);
26    }
```

This uses **ChromaDB** to create 16 collections, there are 5 **Ollama** embedding models and 3 of them support prefixes

main videoprojects / 2024-10-29-embeddings / vectorprep.ts ↑ Top

Code Blame 61 lines (55 loc) · 2.38 KB Raw

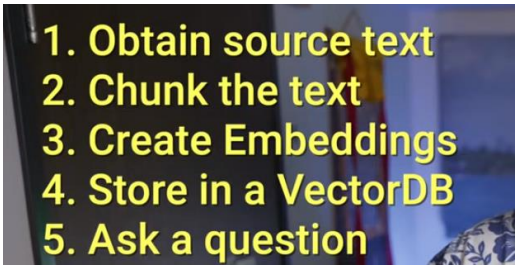
```
14
15
16  for await (const model of models) {
17    try {
18      await client.deleteCollection({ name: `${model.name}-with-questions` });
19    } catch (_error) {
20      console.log(`Collection ${model.name}-with-questions did not exist`);
21    }
22      try {
23      await client.deleteCollection({ name: `${model.name}-without-questions` });
24    } catch (_error) {
25      console.log(`Collection ${model.name}-without-questions did not exist`);
26    }
27    const collectionwithquestions = await client.getOrCreateCollection({
28      name: `${model.name}-with-questions`,
29      metadata: { "hnsw:space": "cosine" },
30    });
31    const collectionwithoutquestions = await client.getOrCreateCollection({
32      name: `${model.name}-without-questions`,
33      metadata: { "hnsw:space": "cosine" },
34    });
35    const sourceembeddings: { source: string; embed: number[] }[] = [];
36    const questionembeddings: { source: string; embed: number[] }[] = [];
37    for await (const line of sourceScriptLines) {
38      const embedresponse = await Ollama.embed({
39        model: model.model,
40        input: `${model.dprefix}${line}`,
41      });
42      sourceembeddings.push({ source: line, embed: embedresponse.embeddings[0] });
43    }
```

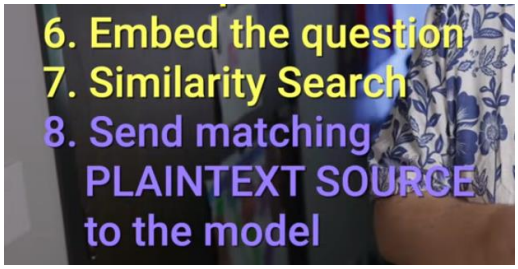
main videoprojects / 2024-10-29-embeddings / vectorprep.ts ↑ Top

Code Blame 61 lines (55 loc) · 2.38 KB Raw

```
33      metadata: { "hnsw:space": "cosine" },
34    });
35    const sourceembeddings: { source: string; embed: number[] }[] = [];
36    const questionembeddings: { source: string; embed: number[] }[] = [];
37    for await (const line of sourceScriptLines) {
38      const embedresponse = await Ollama.embed({
39        model: model.model,
40        input: `${model.dprefix}${line}`,
41      });
42      sourceembeddings.push({ source: line, embed: embedresponse.embeddings[0] });
43    }
44      for await (const q of questions) {
45      const embedresponse = await Ollama.embed({
46        model: model.model,
47        input: `${model.dprefix}${q}`,
48      });
49      questionembeddings.push({ source: q, embed: embedresponse.embeddings[0] });
50    }
51
52    const embeds = sourceembeddings.map((e) => e.embed);
53    const docs = sourceembeddings.map((e) => e.source);
54    const qembeds = questionembeddings.map((e) => e.embed);
55    const qdocs = questionembeddings.map((e) => e.source);
56    const sourceids = docs.map((_, index) => `${model.name}-source${index}`);
57    const qids = qdocs.map((_, index) => `${model.name}-question${index}`);
58    await collectionwithoutquestions.add({ ids: sourceids, embeddings: embeds, documents: docs });
59
60    await collectionwithquestions.add({ ids: [...sourceids, ...qids], embeddings: [...embeds, ...qembeds], documents: [...docs, ...qdocs] });
61  }
```

Documents have been chunked up by paragraphs




```
main [$x!?] +11 -452                                v22.4.0 via v2.0.3
❯ deno --allow-read --allow-net test1.ts
```

```
Model:
nomic
Question:
What is n8n?

Input:
How do I run n8n on my Mac?
```

```
Model:
prefixed-nomic
Question:
What is n8n?

Input:
Luckily there is another way to install n8n. It's with npm. That's the pa
ckage manager for NodeJS. So I can run `npm i -g n8n`. i for install and
g for global. I still have that docker container running, so 'docker comp
ose stop', and then run `n8n`.
```

```
What is n8n?

Input:
How do I run n8n on my Mac?
```

```
Model:
all-minilm
Question:
What is n8n?

Input:
Luckily there is another way to install n8n. It's with npm. That's the pa
ckage manager for NodeJS. So I can run `npm i -g n8n`. i for install and
g for global. I still have that docker container running, so 'docker comp
ose stop', and then run `n8n`.
```

```
Model:
snowflake-arctic
Question:
What is n8n?

Input:
What does GIK make?
```

```
Model:
prefixed-snowflake-arctic
Question:
What is n8n?

Input:
How do I run n8n on my Mac?
```

```
Model:
bge
Question:
What is n8n?

Input:
How do I run n8n on my Mac?
```

All the results aren't correct. Let us see the response when we use docker-compose as below

```
Model:
prefixed-nomic
Question:
How do I install n8n with docker compose?

Input:
So let's get n8n installed using docker compose. If you look online for a
 simple example, you'll hit our first problem. Every example is super com
plicated. It's local, we have one user, we don't need a complicated datab
ase setup or let's encrypt ssl certs. So in the github repo for this vide
o, you'll find a good simple compose file. It specifies the image, a cont
ainer name, ports, some environment info and a volume for persistence. Ru
n `docker compose up -d` and we are set. Let's create a new owner account
. We need a name, email, and password. And then you can create a workflow
.
```

```
Model:
prefixed-snowflake-arctic
Question:
How do I install n8n with docker compose?

Input:
So let's get n8n installed using docker compose. If you look online for a
 simple example, you'll hit our first problem. Every example is super com
plicated. It's local, we have one user, we don't need a complicated datab
ase setup or let's encrypt ssl certs. So in the github repo for this vide
o, you'll find a good simple compose file. It specifies the image, a cont
ainer name, ports, some environment info and a volume for persistence. Ru
n `docker compose up -d` and we are set. Let's create a new owner account
. We need a name, email, and password. And then you can create a workflow
.
```

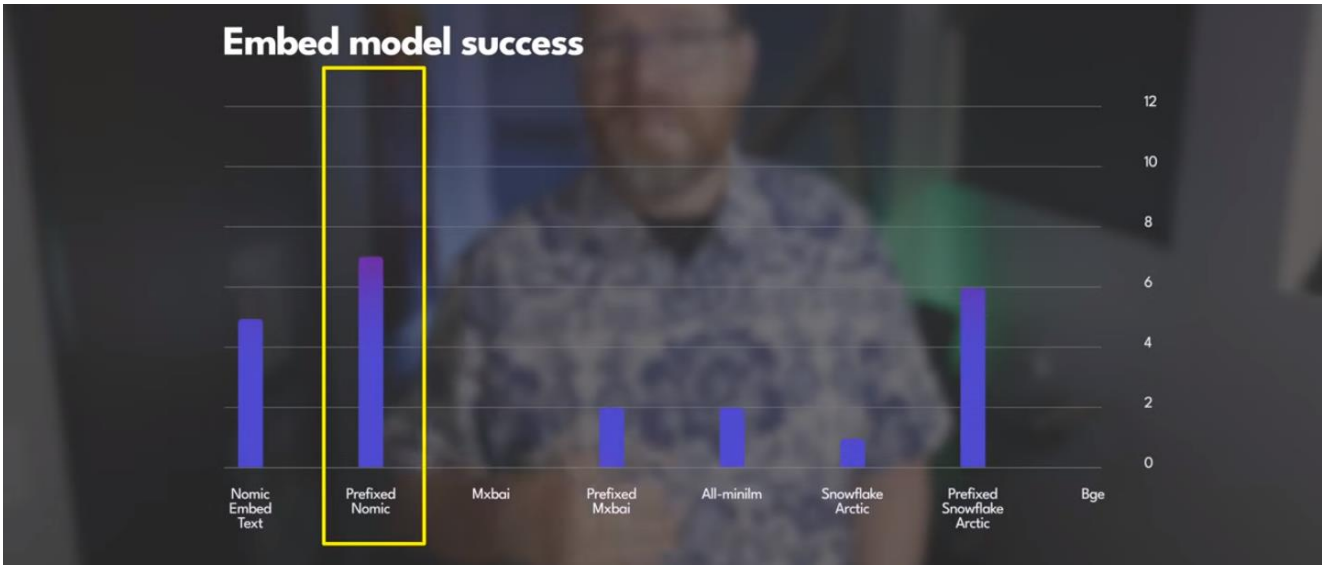
```
Model:
bge
Question:
How do I install n8n with docker compose?

Input:
what is the downside of using n8n with docker compose on the mac?
```

Using docker-compose gives us better results as above

```
Model:
nomic
Question:
How do I run n8n on my Mac?

Input:
What is n8n?
```

```
Model:
prefixed-nomic
Question:
How do I run n8n on my Mac?

Input:
Luckily there is another way to install n8n. It's with npm. That's the pa
ckage manager for NodeJS. So I can run `npm i -g n8n`. i for install and
g for global. I still have that docker container running, so 'docker comp
ose stop', and then run `n8n`.
```

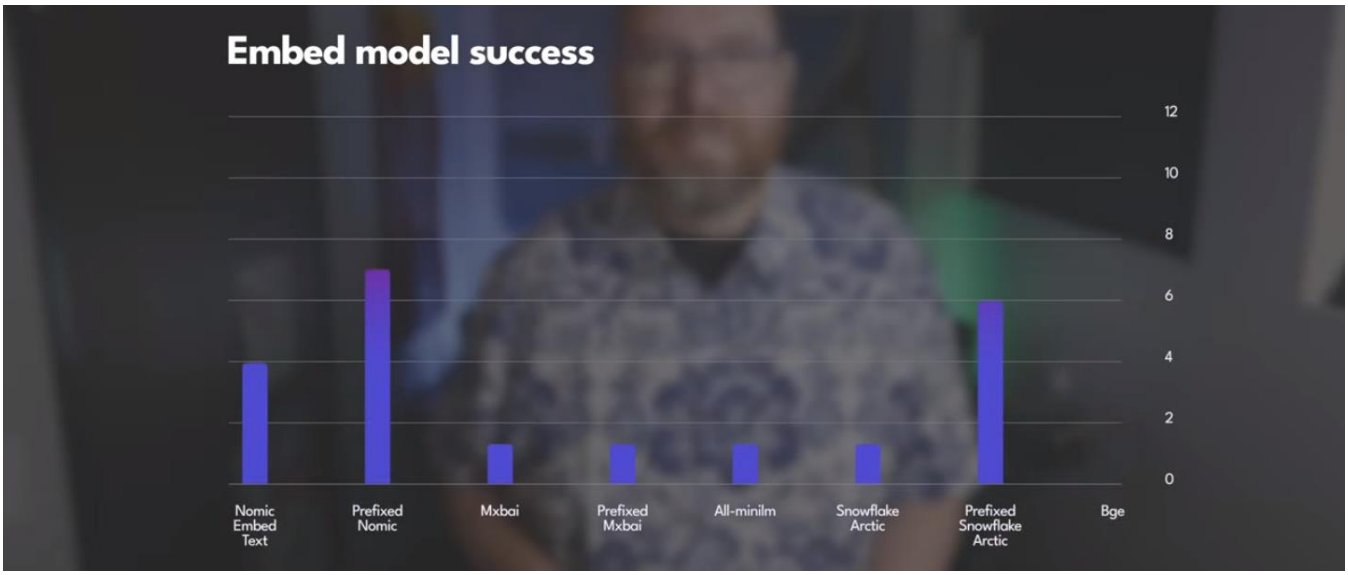
Only nomic gives an okay result

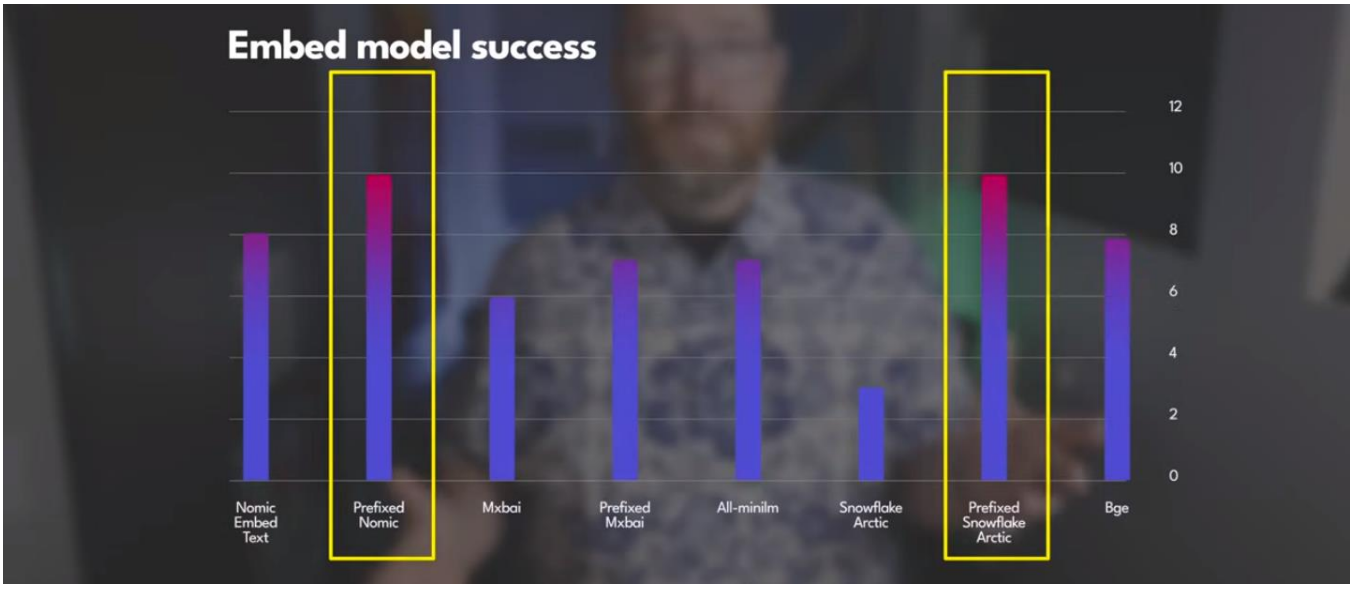

```
DOCUMENT:
${topinputdocs}

QUESTION:
${q}

INSTRUCTIONS:
Answer the users QUESTION using only the DOCUMENT
text above. Ignore all prior knowledge. Keep your
answer ground in the facts of the DOCUMENT. If the
DOCUMENT doesn't contain the facts to answer the
QUESTION return {NONE}. Your response should only
include the answer. Do not provide any further
explanation.`
```

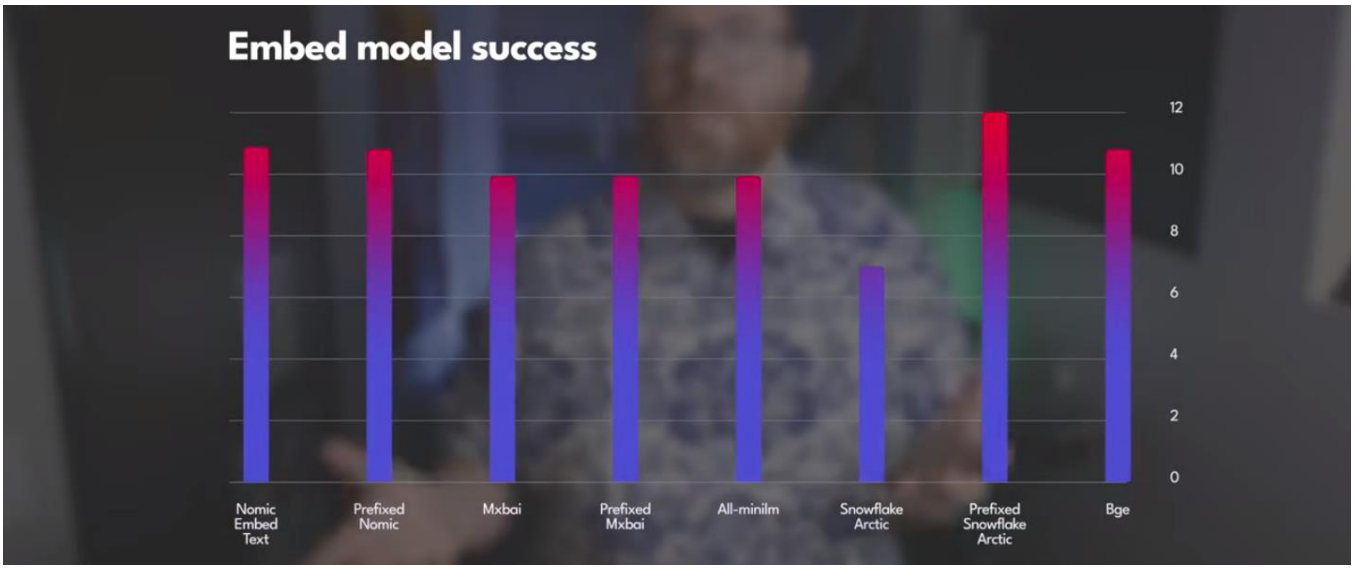
We can try the IBM Granite:8B dense model for the next task


RUNNING
TEST 2


Embed model success
12
10
8
6
4
2
0
Nomic Embed Text
Prefixed Nomic
Mxbai
Prefixed Mxbai
All-minilm
Snowflake Arctic
Prefixed Snowflake Arctic
Bge


RUNNING
TEST 3
Documents:
Ollama is the easiest way to run large language models on your computer,
whether its Mac, Windows, or Linux. And Hugging Face has the largest coll
grab any GGUF
import it int
et GGUF models
g on Ollama.


Embed model success
12
10
8
6
4
2
0
Nomic Embed Text
Prefixed Nomic
Mxbai
Prefixed Mxbai
All-minilm
Snowflake Arctic
Prefixed Snowflake Arctic
Bge




We get the above results when we add more documents to be embedded and used.